

ASHAKids | Auditoría del backend

01. Dictamen ejecutivo

Fecha de corte: 8 de octubre de 2026. Alcance: backend FastAPI, contratos HTTP, persistencia, permisos, configuración y preparación para despliegue. Auditoría técnica con implementación y pruebas; no certificación clínica, pentest ni aprobación de producción.

Resultado: base funcional implementada y verificada; despliegue público todavía condicionado. No conviene rehacer el backend desde cero. La estructura existente permite extender funciones por dominio y conservar el esquema de datos.

Indicador	Resultado comprobado
Operaciones en OpenAPI	32: 29 de autenticación/dominio y 3 de sistema
Ampliación realizada	24 operaciones nuevas respecto de las 8 previas
Pruebas automatizadas	63 correctas; 5 omitidas deliberadamente; 0 fallidas
Advertencias	19 de deprecación en la suite heredada
PostgreSQL aislado	Versión 18; escrituras, consultas, rollback y concurrencia comprobados
Supabase compartido	PostgreSQL 17.6; conectividad y columnas de 16 modelos comprobadas sin extraer registros
APIs rotas observadas	Ninguna en los recorridos ensayados; no equivale a ausencia total de defectos

Ya disponible por API: administrar usuarios, registrar y editar pacientes, asignar tratamientos, reservar/reprogramar/confirmar/cancelar citas, registrar/iniciar/cerrar sesiones y guardar/consultar reportes. Se conservan login, logout y perfiles por rol.

Lo que no está terminado: integración de estas operaciones con las pantallas, seguridad operativa de producción, trazabilidad clínica y módulos de pagos, mensajes, actividades, juegos e IA. La interfaz continúa simulando varios flujos.

No se crearon usuarios ni pacientes reales en Supabase. No se cambiaron sus tablas, contraseñas ni políticas. El backend local quedó actualizado; no se hizo commit, push ni despliegue remoto.

02. Arquitectura y cambios realizados

Se conserva React -> HTTP/JSON -> FastAPI -> SQLAlchemy asíncrono -> PostgreSQL. Supabase proporciona la base de datos; no se introdujo Supabase Auth ni acceso directo desde React.

Capa	Responsabilidad y archivos principales
api/v1	usuarios.py, pacientes.py, citas.py y sesiones.py: rutas y respuestas HTTP
api/contracts.py	Sesión DB por petición, identidad y paginación compartidas
schemas	Contratos de usuarios y clínica; rechaza campos extra, fechas inválidas y entradas fuera de límite
services	Permisos por recurso, transiciones, conflictos horarios y persistencia
models/clinica.py	Expediente, Tratamiento, Reserva, Sesion y ReporteSesion sobre tablas existentes
core	Configuración, Argon2id, motor asíncrono y manejo seguro de errores DB
tests y scripts	Pruebas aisladas y auditoría de solo lectura reproducible

Las rutas /citas guardan en reservas; no se creó una segunda tabla. El identificador de respuesta sigue siendo id_reserva. Registrar tratamiento puede abrir un expediente vacío: no inventa diagnósticos ni historias clínicas.

Autorización: ADMIN administra usuarios y asignaciones. PADRE accede a su propia familia. TERAPEUTA ve pacientes asignados y opera únicamente sus citas/sesiones. Los accesos a registros ajenos se ocultan con 404; permisos insuficientes producen 403. Los roles combinados no convierten una familia ajena en propia.

Integridad: commit antes del éxito HTTP; rollback ante error; 409 para duplicados/conflictos y 503 para indisponibilidad. Bloqueos de paciente y terapeuta evitan solapamientos concurrentes por la API. Una reserva solo puede tener una sesión.

Autenticación: cuenta sin roles válidos no obtiene sesión. Rol principal determinista ADMIN > TERAPEUTA > PADRE. Hashing fuera del hilo del bucle asíncrono. Desactivar usuario o cambiar contraseña revoca sus sesiones.

Decisiones y límites operativos: docs/decisions/0003-backend-persistencia-base.md. Estas reglas técnicas deben revisarse con el equipo responsable antes de uso clínico.

03. APIs funcionando: identidad y pacientes

Todas las rutas de esta página llevan el prefijo /api/v1. OK significa respuesta exitosa y comportamiento ensayado con PostgreSQL local aislado; no un login con identidades reales de Supabase. Las consultas devuelven como máximo 100 registros por página mediante limit y offset.

Método y ruta	Éxito	Permiso / alcance
POST /auth/login	200	Credenciales válidas; cookie y sesión persistente
POST /auth/logout	200	Revoca token y elimina cookie
GET /auth/me	200	Identidad con sesión vigente
GET /padres/me	200	PADRE; perfil tutor
GET /terapeutas/me	200	TERAPEUTA; perfil profesional
GET /admin/me	200	ADMIN; perfil administrativo
GET /usuarios	200	ADMIN; listado paginado sin hashes
POST /usuarios	201	ADMIN; usuario, rol y perfil en una transacción
GET /usuarios/{key}	200	ADMIN; detalle seguro e identificadores de perfil
PATCH /usuarios/{key}	200	ADMIN; datos, contraseña o activación
GET /pacientes	200	Listado filtrado por familia/asignación
POST /pacientes	201	PADRE propio o ADMIN indicando tutor
GET /pacientes/{key}	200	Paciente visible para la identidad
PUT /pacientes/{key}	200	PADRE propio o ADMIN; reemplaza datos básicos
DELETE /pacientes/{key}	200	Baja lógica; no elimina el historial
GET /pacientes/{key}/tratamientos	200	Familia, profesional asignado o ADMIN
POST /tratamientos	201	ADMIN; asignación explícita paciente/profesional

Las contraseñas de usuarios nuevos se exigen entre 12 y 128 caracteres. Esta regla no cambia automáticamente claves antiguas. No existe registro público ni cambio de rol/tutor desde los formularios anteriores.

Las respuestas no incluyen hashes de contraseña ni tokens de sesión. Se conserva el contrato detail de errores para compatibilidad con el cliente HTTP del frontend.

04. APIs funcionando: citas, sesiones y sistema

Las rutas de dominio usan /api/v1. Las tres rutas de sistema no llevan prefijo.

Método y ruta	Éxito	Permiso / alcance
GET /citas	200	Solo citas visibles para la identidad
POST /citas	201	Familia, terapeuta asignado o ADMIN; tratamiento activo
GET /citas/{key}	200	Participantes autorizados o ADMIN
PUT /citas/{key}	200	Reprograma y devuelve a PENDIENTE
PATCH /citas/{key}/estado	200	Confirmar: profesional/ADMIN; cancelar: participantes
GET /sesiones	200	Filtradas mediante citas autorizadas
POST /sesiones	201	Profesional/ADMIN; cita confirmada y sin otra sesión
GET /sesiones/{key}	200	Participantes autorizados o ADMIN
POST /sesiones/{key}/iniciar	200	Profesional/ADMIN; horario ya iniciado
POST /sesiones/{key}/cerrar	200	Profesional/ADMIN; valida transición y asistencia
GET /sesiones/{key}/reporte	200	Participantes; 404 si aún no hay reporte
PUT /sesiones/{key}/reporte	200	Profesional/ADMIN; sesión iniciada o finalizada
GET /	200	Servicio y versión
GET /health	200	Proceso activo; no comprueba PostgreSQL
GET /health/ready	200	Ejecuta SELECT 1; caída de DB produce 503

Estados de cita: PENDIENTE -> CONFIRMADA -> COMPLETADA. Se permite CANCELADA antes de registrar sesión. Estados de sesión: PROGRAMADA -> EN_CURSO -> FINALIZADA. La inasistencia solo se registra después del final previsto; no inicia una sesión ficticia.

Errores esperados, no APIs rotas: 401 sin sesión/expirada; 403 por rol u origen; 404 por recurso ausente/ajeno; 409 por horario ocupado, duplicado o transición inválida; 422 por contrato inválido; 503 por DB no disponible.

En localhost se comprobaron 200 en raíz, health, readiness y OpenAPI; auth/me anónimo devolvió el 401 esperado. El inventario OpenAPI servido coincide con el código nuevo.

05. APIs no operativas y capacidades faltantes

No se detectó una operación implementada rota en los casos ensayados. Lo siguiente permanece ausente o incompleto; NO debe presentarse como funcional porque exista una pantalla. Los nombres entre barras son familias propuestas, no contratos implementados.

Capacidad	Estado actual	Trabajo pendiente
Pantallas de clínica	API lista; UI simulada	Conectar pacientes, agenda, sesiones y reportes con estados de error/carga
Recuperar contraseña	Sin API	/auth/recuperacion y restablecimiento con token temporal y canal seguro
Roles y perfiles editables	Parcial	Gestión de roles/perfiles y cambios de tutor con trazabilidad
Expedientes clínicos	Solo apertura vacía	Contratos de historia clínica, diagnóstico, consentimiento y acceso profesional
Tratamientos completos	Crear y listar	Editar, cerrar, reasignar y consultar detalle con historial
Disponibilidad profesional	Sin API	Agenda laboral, bloqueos, vacaciones y zona horaria operativa
Videollamadas	Sin integración	Proveedor real, generación de reunión y permisos; no se generan links ficticios
Pagos	Sin API	Órdenes, estados, webhooks verificados e idempotencia; modelo por definir
Actividades y objetivos	Tablas sin endpoints	Asignación, avance, resultados y permisos por paciente
Mundos, niveles y logros	Tablas/modelos parciales	Catálogo y resultados persistentes; no equiparar mocks con progreso real
Evaluación IA	Tabla sin servicio	Procesamiento, consentimiento, revisión humana y límites clínicos
Mensajería y adjuntos	Tablas sin endpoints	Conversaciones, archivos, autorización y notificaciones
Métricas y auditoría	Sin API ni escritura auditada	Agregados autorizados, registro de cambios y versiones de reportes

No se probaron credenciales reales de los tres roles mediante login HTTP contra Supabase en esta auditoría. Las pruebas nuevas usan identidades sintéticas propias; no demuestran que un código de demostración exista en la base compartida.

Los valores de estado heredados que no correspondan al contrato nuevo requieren revisión, no normalización automática de registros reales. El script SQL base y los modelos no sustituyen migraciones versionadas.

06. Seguridad y riesgos priorizados

P0: resolver antes de exposición pública. P1: imprescindible antes de operar con datos clínicos reales. P2: mejora técnica priorizada tras estabilizar el MVP.

Prioridad	Hallazgo	Acción necesaria
P0	Se compartieron credenciales de infraestructura en el chat	Rotar contraseña DB por canal seguro; no se rotó ni se reprodujo en este informe
P0	Semillas heredadas incluyen claves débiles y ON CONFLICT que cambia contraseñas	No ejecutar semillas en producción; revisar cuentas y eliminar accesos de demostración
P1	No hay limitación distribuida de intentos de login	Aplicar rate limiting y alertas de abuso en API/proxy antes de publicar
P1	Reportes clínicos editables sin historial	Registrar actor/fecha/cambio y definir política de cierre y consentimiento
P1	Conexión DB con BYPASSRLS	Crear rol de aplicación de mínimo privilegio y probar permisos antes de cambiar credenciales
P1	Flujos UI nuevos aún sin integración	Conectar endpoints y ejecutar prueba completa en staging con los tres roles
P1	Sin migraciones incrementales ni restauración ensayada	Versionar cambios; backup y recuperación verificados; no ejecutar CREATE TABLE sobre la BD existente
P1	Producción HTTPS/CORS/cookies no ensayada en hosting	Usar mismo sitio HTTPS y validar login, recarga, expiración y logout reales
P2	Consultas de usuarios realizan lecturas de perfiles por fila	Reducir N+1 si crece el volumen; medir antes de optimizar
P2	Dependencias por rangos y pruebas con APIs deprecadas	Crear lock reproducible, análisis de vulnerabilidades y actualizar cliente de pruebas

Controles comprobados: Argon2id, token opaco almacenado como hash, cookie HttpOnly y Secure en producción, Origin autorizado para escrituras con cookie, Cache-Control no-store, baja lógica, paginación, bloqueo de solapamientos, rollback y errores sin SQL de conexión.

Supabase mostró RLS habilitada en las 26 tablas públicas y cero políticas públicas. Existen grants para anon/authenticated; esto por sí solo no prueba exposición de filas. El usuario de conexión omite RLS, por lo que los permisos de FastAPI son críticos. No se hicieron pruebas ofensivas de extracción de registros.

No se realizó pentest, ensayo de carga, auditoría de cumplimiento ni escaneo de vulnerabilidades de dependencias. pip check solo comprobó compatibilidad de paquetes instalados.

07. Evidencia de validación y límites

Comprobación	Resultado	Evidencia reproducible
Suite completa pytest	63 passed, 5 skipped	tmp/backend-audit-tests.xml
Integración clínica	PostgreSQL real local	tests/test_clinical_api.py y test_clinical_edges.py
Persistencia	Lectura en sesión DB nueva	Paciente y reporte almacenados; no solo respuesta HTTP
Concurrencia	201 + 409	Dos reservas simultáneas; dos sesiones para la misma cita
Permisos	Casos positivos y negativos	Familia ajena, terapeuta no asignado, rol combinado y anónimo
Sesión	Correcto en BD aislada	Login de tres roles, /me, logout, expiración y revocación
Fallo DB y commit	503 / 409	Conexión rechazada y error de commit inyectados; no falso éxito
Supabase	Solo lectura, OK	16 SELECT LIMIT 0; versión 17.6; metadatos RLS
HTTP local actualizado	OK	/health/ready 200; OpenAPI coincide; anónimo 401
Sintaxis / dependencias	Correcto	compileall y pip check

Las cinco pruebas omitidas pertenecen a la suite heredada que escribe sesiones en la base compartida. Ahora requieren una variable de autorización explícita. No se cuentan como aprobadas. Las pruebas locales nuevas ejercitan autenticación real con Argon2id y PostgreSQL sin usar esos usuarios.

Las 19 advertencias son una de compatibilidad Starlette/TestClient-httpx y 18 por cookies enviadas por petición en pruebas heredadas. No se silenciaron. No hubo errores de conexiones sin cerrar en la ejecución final observada.

Reproducibilidad: backend/scripts/audit_readonly.py inventaría las 32 operaciones y consulta columnas sin leer registros clínicos. tmp/backend-api-evidence.json agrupa método/ruta/estado observado sin cuerpos, cookies ni contraseñas. Las evidencias de este corte se copian a docs/evidence/backend-2026-10-08/.

La BD aislada se creó desde backend/scripts/db_creation.sql en PostgreSQL 18. Supabase ejecuta 17.6: no son el mismo entorno. La comprobación de columnas no demuestra igualdad de todas las restricciones, permisos, triggers ni rendimiento bajo carga.

08. Cierre y ruta de despliegue

El objetivo inmediato ya tiene una base verificable: los flujos centrales guardan y consultan datos por API, con permisos por recurso y pruebas de errores. No se declara la plataforma lista para producción ni un porcentaje global de avance: faltan criterios acordados para medir todos los módulos del producto.

Antes del sábado 10 de octubre:

1. Acordar el MVP: usuarios, pacientes, agenda y sesiones con reporte. Ocultar o etiquetar como demostración pagos, IA, juegos y mensajería que no estén integrados.
2. Resolver P0: rotar secretos y retirar cuentas/semillas débiles por un canal autorizado. Nunca incorporar contraseñas a VITE, Git, reportes o capturas.
3. Conectar el frontend a los nuevos contratos y eliminar los mocks solo en los flujos efectivamente integrados. No basta con abrir una ruta.
4. Preparar staging con HTTPS, CORS exacto, mismo sitio para cookies, límite de intentos, backups y rol DB restringido. Usar /health/ready para disponibilidad.
5. Validar con usuarios autorizados de cada rol: login, recarga, crear paciente, asignar tratamiento, reservar, confirmar, iniciar, reportar, cerrar y logout; comprobar 403/404 entre familias.
6. Autorizar producción solo después de una prueba de restauración y revisión de consentimiento/trazabilidad si se usarán datos clínicos reales.

Ejecución local: desde backend, instalar requirements-dev.txt; ejecutar `python -m pytest -q`. Las pruebas PostgreSQL nuevas requieren `ASHAKIDS_TEST_DATABASE_URL` con host `localhost/127.0.0.1` y una BD descartable cuyo nombre comience por `ashakids_test_`. La fixture elimina datos únicamente en esa BD explícita. Sin variable, se omiten: no presentar esa ejecución como integración aprobada.

Levantar API: `python -m uvicorn app.main:app --host 127.0.0.1 --port 8000`. Para puertos DB usar `DB_PORT`; reservar PORT para HTTP. Documentación interactiva en <http://localhost:8000/docs>. Los procesos locales no constituyen un despliegue remoto.

Fuentes de evidencia: código actual de backend/app; scripts/db_creation.sql; tests; OpenAPI servido; metadatos PostgreSQL consultados el 8 de octubre; servicios actuales de frontend. Referencia técnica: FastAPI, Dependencies with yield, <https://fastapi.tiangolo.com/tutorial/dependencies/dependencies-with-yield/> (scope=function para finalizar transacción antes de respuesta).

Recomendación final: continuar esta base modular. El trabajo siguiente debe centrarse en integración y preparación segura del despliegue, no en rehacer el backend.